

스마트 포인터와 자동 메모리 관리 보고서

C++ 프로그래밍

20230841

인공지능학부 인공지능전공

최준수

스마트 포인터와 자동 메모리 관리

■ 조사 목적

본 조사는 국민대학교 소프트웨어융합대학 C++프로그래밍(0192809-01)를 수강하며 학습한 C++의 스마트 포인터 개념을 이해하고, 동적 메모리 관리 문제를 어떻게 개선하고 있는지를 분석하는 것을 목적으로 한다.

또한, C++의 Reference Count의 동작 원리를 이해하고, Java Virtual Machine 기반의 메모리 관리 방식과 비교하여 C++의 자동 메모리 변환 방식의 장단점을 이해한다.

■ 본문

○ 스마트 포인터(Smart Pointer)의 개념과 필요성

최신 C++ 프로그래밍에서 표준 라이브러리에는 프로그램이 메모리 및 리소스 누수로 부터 자유롭고 예외로부터 안전한지 확인하는데 사용되는 스마트 포인터(Smart Pointer)가 포함되어있다.¹⁾ 즉, 스마트 포인터는 메모리 관리를 자동화하기 위해 만들어진 C++의 표준 라이브러리이다. 스마트 포인터의 등장 배경과 그 필요성을 이해하기 위해서는 기존 일반 포인터(Raw Pointer)가 가지고 있던 단점과 그 위험성을 이해할 필요가 있다.

우선 포인터를 이해해 보자, 마이크로소프트 공식 문서에서는 포인터를 다음과 같이 설명하고 있다. “포인터는 개체의 메모리 주소를 저장하는 변수입니다.”²⁾ 그렇다, C언어에서 특정 함수 내의 지역 변수는 함수 종료와 함께 사라진다. 하지만, 이 변수의 주소 값을 포인터로 전달하게 된다면, 함수 외부에서도 해당 변수의 메모리 공간에 직접 접근이 가능하다. 그렇다, 지역 변수에 접근하기 위해 사용한다는 점에서 Python 언어의 레퍼런스와 큰 범위에서 개념적 뿌리가 같다고 할 수 있다. 사실, 등장 시기를 보면, Python 언어의 레퍼런스라는 개념이 C 언어의 포인터 개념에서 착안했다는 것이 더 맞을 수 있다.

하지만, Python 언어의 레퍼런스와 100% 동일하다고 할 수는 없다. Python 언어의 레퍼런스와 C 언어의 포인터의 차이를 수업시간에 학습하며 나는 항공기를 만드는 양대 산맥 보잉(Boeing)과 에어버스(AirBus)의 항공기 설계 철학 차이와 비슷하다고 생각했다. 보잉은 기계보단 인간의 판단이 우선한다는 철학으로 항공기를 제작하고, 에어버스는 인간은 항상 실수 할 수 있기 때문에 시스템이 보호해야한다 라는 철학을 가지고 있다.

C언어의 포인터는 그 변수의 실제 메모리 주소(ex: 0x1004)를 저장한다. 즉, 메모리 주소에 직접 접근이 가능하기 때문에 Pointer에 +1을 하여 다음 메모리 칸으로 이동하는 등의 트릭이 가능하다. 이를 포인터 연산이라고 하고 하드웨어를 직접 제어할 수 있을 만큼 강력하지만, 잘못된 주소를 건들면 프로그램이 즉시

1) Microsoft. “스마트 포인터 (최신 C++).” Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/smart-pointers-modern-cpp?view=msvc-170>

2) Microsoft. “포인터 (C++).” Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/pointers-cpp?view=msvc-170>

종료될 수 있다. 사용자에게 넓은 범위에서 선택의 자유를 준다는 점에서 C언어의 포인터는 보잉의 항공기 설계 철학과 매우 유사하다고 느꼈다. Python의 레퍼런스는 내부적으로 주소를 가리키지만, 개발자가 그 주소 값을 숫자로 취급하거나, 뺄 수 없다. 나를 포함해서, Python을 첫 언어로 학습한 사람을 오히려 주소 값을 사칙 연산으로 접근할 수 있다는 발상 자체가 신기하다고 느낄 것이다. Python은 레퍼런스를 오직 객체에 접근하기 위한 안전한 핸들 역할만 수행한다는 점에서 에어버스의 항공기 설계 철학과 유사하다고 느꼈다.

또 다른 차이로는 바로 가바지 컬렉터(Garbage Collector)의 유무이다. Python을 코딩해본 사람이라면 레퍼런스를 스스로 방출한 기억은 없을 것이다. 그 이유는 Python 레퍼런스는 기본적으로 가바지 컬렉터가 뒤를 봐주고 있기 때문이다. 특정 객체를 가리키는 레퍼런스의 개수(Reference Count)가 0이 되면 Python이 알아서 메모리를 수거해주기 때문에, Python 레퍼런스를 방출하지 않았어도 그 프로그램이 잘 실행됐던 것이다. 눈치 빠른 사람은 이렇게 Python의 가바지 컬렉터를 칭찬한다는 것을 보고 바로 알 수 있을것이다. 그렇다, C언어 포인터는 그 가바지 컬렉터가 없다. 개발자가 malloc(동적할당)이나 new로 메모리를 할당 했다면, 반드시 free나 delete로 직접 해제해야한다.³⁾ 이를 잊으면 메모리 누수(Memory Leak)가 발생한다. 더 나아가서 개발자가 실수로 포인터가 담고 있는 주소를 잃고 그로 인해 할당 받은 자원을 찾지도 못하고 해제하지도 못하게 되거나, 이미 해제한 것을 이중으로 해제하는 등의 문제도 있다.⁴⁾ 이것이 기존 일반 포인터(Row Pointer)가 가지고 있던 단점과 그 위험성이다.

개발자들은 C++ 에서 그러한 문제를 해결하기 위해 객체 스스로 자원을 관리 가능하도록 클래스를 만들어 사용했었는데, 이것이 스마트 포인터의 원조이다.⁵⁾ 객체가 스스로 자원을 관리하는 의미는 멤버함수를 통해 할당 받은 자원을 소멸자에서 그 자원을 방출한다는 의미이다. 즉, 앞서 이야기한 에어버스의 항공기 설계 철학처럼 인간은 항상 실수 할 수 있기 때문에 시스템이 보호하게 된 것이다.

○ Reference Count의 개념과 원리

앞서, Python 언어의 레퍼런스의 특징을 설명하면서 Reference Count에 대해서 언급했었다. Reference Count를 우리말로 직역하자면 ‘참조 횟수’ 정도로 직역 가능하다. 즉, 특정 메모리 자원을 가르키고(Pointing) 있는 객체의 개수를 나타낸다. 단순히 지금 이 데이터를 몇 명이 공유(shared_ptr)하고 있는지? 를 나타낸 기록이라고 이해하면 편하다.

Shared_ptr은 여러 상황에서 카운드가 자동으로 올라가거나 줄어든다. Shared_ptr이 증가하는 경우로는 `shared_ptr<int> p2 = p1;` 처럼 기존의 포인터

3) Microsoft. “new 및 delete 연산자.” Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/new-and-delete-operators?view=msvc-170#--the-delete-operator>

4) 위키독스. “02.02.02. 스마트포인터(Smart Pointer).” C++ 이야기(A Stroy of C++). <https://wikidocs.net/117594>

5) 위키독스. “02.02.02. 스마트포인터(Smart Pointer).” C++ 이야기(A Stroy of C++). <https://wikidocs.net/117594>

를 복사할 때 카운트가 올라간다. 또, 함수의 인자로서 `shared_ptr` 값을 넘기면 (pass by value) 카운트가 올라간다. 당연히 감소하는 경우도 존재한다. 함수가 끝나거나 코드 블록(Scope)을 벗어나면서, `shared_ptr` 객체가 소멸될 때 카운트가 내려간다. 포인터 변수에 다른 주소를 넣거나 `reset()` 함수를 호출하면 그 역시 기존에 가리키던 자원에 대한 카운트가 내려간다.⁶⁾ 이러한 방식으로 참조 횟수를 관리하는 방법은 매우 편리하고 장점 역시 뚜렷하지만, 당연히 한계도 존재한다. 우선 Smart Pointer 내부적으로 작동하기 때문에 프로그래머가 언제 `delete`를 호출할 지 고민하지 않아도 된다. 특히, 코드 중간에 에러가 터져도 소멸자가 호출되면서 메모리가 확실히 해제가 된다. 단점 역시 여러 방향으로 있지만, 순환 참조가 Reference Count의 아킬레스건이다. 예를 들어보자, 만실인 교실 A와 교실 B가 있다고 하자, 교실 A속 학생 한 명이 교실 B로 옮길려고 하고, 그와 동시에 교실 B속 학생 한 명이 교실 B에서 교실 A로 옮길려고 한다. 이때, 두 학생이 서로 “네가 나와야 내가 나간다”라며 버티는 상황이다. 이 상황을 순환 참조로 비유하면, 교실 A(객체 A)는 “나는 `shared_ptr`로 교실 B를 가리키고 있어. 즉, B가 살아있는 한 나도 이 메모리에서 나가지 않을 거야.”라고, 교실 B(객체 B) 역시 “나도 `shared_ptr`로 교실 A를 가리키고 있어. A가 살아있는 한 나도 절대 나가지 않아.”라고 주장하고 있는 것이다. Main 함수는 두 객체를 모두 방출 하고 싶어도, 두 객체가 서로 순환 참조 하고 있기 때문에 영원히 메모리를 점유하고 있는 상황인 것이다.

○ Java Virtual Machine(JVM)과의 연관성

프로그램을 개발하다 보면 필연적으로 유효하지 않은 메모리인 가비지(Garbage)가 발생하게 된다. 위에서 설명한 것처럼 C언어에서는 `free()`와 `delete()` 등의 함수를 통해 직접 메모리를 해제해 준다. 하지만, Java와 python 등의 조금 더 높은 레벨의 고등언어에서는 개발자가 직접 메모리를 해제해주는 일이 없다. 그 이유는 Java 언어는 JVM에서 참조 횟수를 세는 Reference Counting 대신 가비지 컬렉션(Garbage Collection)을 채택해 자동으로 불필요한 메모리를 알아서 정리해 주기 때문이다.⁷⁾ C++언어의 스마트 포인터와 Java의 JVM 가비지 컬렉션 모두 메모리 관리 자동화라는 공통점이 있지만, 그 작동방식에는 큰 차이가 있다.

Reference counter는 참조 횟수를 세는 것으로 메모리 관리를 한다고 했다. JVM의 가비지 컬렉터는 참조 횟수를 세는 대신, 이 객체에 다시 접근할 방법이 있는가를 확인한다. JVM의 가비지 컬렉션은 GC Roots 라는 곳에서 시작한다. GC Roots를 기준점으로 하여 여러 객체에 도달하려 시도한다. 이때, 도달 가능한 객체들을 Reachable로, 어떠한 경로로도 도달할 수 없는 객체(가비지)를 Unreachable로 구분한다.⁸⁾ 이와 같은 방식은 C++ 언어 Reference counter의 치명

6) 모두의 코드. “13 - 2. 자원을 공유할 때 - `shared_ptr` 와 `weak_ptr`.” 씹어먹는 C++ <https://modoocode.com/252>

7) 망나니개발자. “[Java] Garbage Collection(가비지 컬렉션)의 개념 및 동작 원리 (1/2)” 티스토리 <https://mangkyu.tistory.com/118>

8) Oracle. “HotSpot Virtual Machine Garbage Collection Tuning Guide.” Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/javase/21/gctuning/introduction-garbage-collection-tuning.html>

적 단점으로 꼽았던, 순환참조 문제를 아주 손쉽게 해결할 수 있다. 순환 참조가 발생하는 이유는 서로가 서로를 참조하고 있기 때문에, 두 객체가 어디에서도 사용되지 않고 있지만, 참조 Counting이 감소하지 않는 문제가 발생하여 메모리 관리를 하지 않는다는 것에서 출발 했다. 하지만, JVM의 가비지 컬렉터는 아무리 사용되지 않는 두 객체가 서로를 참조하고 있다 하더라도, 두 객체가 사용되지 않고 있기 때문에 GC Roots에서 접근 불가능하기 때문에 메모리 관리가 가능하다.

개발자 관점에서 봤을 때, Java의 JVM 가비지 컬렉션 방식과 C++ 언어의 스마트 포인터 두 방식 중 어떠한 한 가지 방식이 월등히 우월하다고 말하기는 어렵다. JVM의 메모리 관리를 사용한다는 것은 생산성과 안정성을 선택하는 길이 될 것이다. 특히, 개발자가 모든 메모리를 하나하나 세밀하게 관리하지 않아도, JVM 차원에서 관리를 해준다면 생산성 측면에서 매우 큰 도움을 줄 것이다. 하지만, 모든 개발자가 시스템 자원이 풍부한 디바이스를 대상으로 개발을 할 수는 없다. JVM은 기본적으로 Virtual Machine 이기 때문에 시스템 자원이 부족한 디바이스를 대상으로 한 개발에 있어서 로우레벨의 메모리까지 접근 가능한 C 언어보다 접근성이 좋지 못하다. 즉, C++ 스마트 포인터는 변수가 범위를 벗어나자마자 자원이 반납이 되는 등의 즉각적인 반응이 가능하기 때문에 메모리뿐만 아니라 파일 핸들이나 네트워크 소켓, 임베디드 시스템 같은 제한된 리소스를 대상으로 할 때 매우 유리하다.

○ 메모리 자동반환의 장단점

메모리 자동반환을 해주는 언어(Java, Python etc)를 프로그래밍을 해본 입장에서 단언할 수 있는 메모리 자동반환의 가장 큰 장점은 ‘로직 자체에 집중할 수 있다’이다. 자원의 생명 주기를 시스템에 맡김으로 코드가 매우 간결해 지고, “언제 이걸 delete 해주지?”와 같은 고민을 덜 함으로서, 프로그래머가 작성해야 할 로직 자체에 집중할 수 있게 된다. 그의 연장선상으로 메모리 누수 현상 역시 획기적으로 감소한다. 프로그램이 복잡해지고, 일명 스파게티 코드가 될 수록, 프로그래머가 이해하고 관리해야 할 객체들은 기하급수적으로 증가한다. 즉, delete 호출을 누락할 가능성이 증거한다는 것이다. 하지만, 자동 반환의 경우 객체가 더 이상 필요하지 않을 시점으로 스스로 계산하여 회수하기 때문에, 메모리 누수 현상으로 부터 보다 더 자유롭다. 9)

단점역시 존재한다. 바로 ‘성능 오버헤드’이다. 내가 가장 존경하는 경제학자 밀턴 프리드먼 선생님께서는 이런 말씀을 하셨다. “공짜 점심은 없다.” 어떠한 이익을 얻기 위해서는 그에 상응하는 대가를 치뤄야 한다는 것이다. 메모리 자동 반환 역시 동일하다. 바로 성능 오버헤드가 발생한다. 참조 횟수를 관리하기 위한 추가 메모리와 연산 혹은 가비지 컬렉터가 쓰레기를 탐색하기 위해 소비하는 CPU자원은 프로그램의 순수 실행 성능을 소폭 저하시킨다. 또, 제어의 어려움 및 예측 불가능성도 존재한다. VM과 같은 GC 방식에서는 메모리가 '정확히 언제' 해

9) 원정. “[C++ 스마트 포인터(Smart Pointer)란? (Raw Pointer와의 차이).” 원정의 블로그 <https://wonsjung.tistory.com/574>

제될지 알 수 없다.

그럼 이런 단점들이 존재하는 자동반환 방식이 항상 최선인가? 대다수의 상황에서는 그렇지만, 또 그렇다고 모든 상황에서는 또 아니다. 일반적인 개발환경에서는 자동반환이 압도적으로 유리하다. 약간의 성능 손실보다 개발의 생산성을 향상시키는 것이 더 가치있는 행위이기 때문이다. 하지만, 극한의 성능이 필요한 분야가 있을 수 있다. 운영체제의 커널, 실시간 하드웨어 제어 분야 등에서는 여전히 수동 관리나 정밀한 스마트 포인터 설계가 요구된다.

결론적으로 현대 프로그래머에게 요구되는 역량은 "무조건 수동으로 하느냐 자동으로 하느냐"가 아니라, 내가 개발하는 프로그램이 돌아갈 하드웨어적 환경에 맞는 방식으로 개발할 수 있는 능력이 중요하다고 할 수 있다.

■ 결론

본 조사를 통해 C++ 스마트 포인터가 하이레벨 언어의 편의성과 로우레벨 언어의 제어력을 잇는 핵심적 가교임을 확인하였다.

첫째, 설계 철학의 진화를 체감했다. 과거 개발자의 완벽한 통제에 의존하던 방식에서 벗어나, 시스템이 실수를 예방하는 구조로 변화함으로써 현대 소프트웨어의 안정성이 비약적으로 향상되었음을 이해했다.

둘째, 메모리 관리 방식의 트레이드오프를 명확히 파악했다. C++의 Reference Counting 방식은 즉각적인 자원 반납이 가능하나 순환 참조에 취약하며, JVM의 가비지 컬렉션은 순환 참조를 해결하는 대신 성능 오버헤드와 비결정적 해제 시점이라는 비용을 치른다.

셋째, 맥락에 맞는 기술 선택의 중요성을 깨달았다. 임베디드나 실시간 제어 분야에서는 C++의 정밀한 관리가, 대규모 서비스 개발에서는 자동 관리의 생산성이 각각의 강점을 가진다.

결론적으로, 현대 프로그래머에게 요구되는 핵심 역량은 특정 기술의 맹목적 사용이 아니라, 시스템 환경과 목적에 따라 최적의 메모리 관리 전략을 설계하고 선택할 수 있는 기술적 판단력이다. 이번 조사는 단순한 문법 습득을 넘어 자원 관리의 본질적 가치를 고찰하는 유익한 계기가 되었다.

■ 참고자료

- 1) Microsoft. "스마트 포인터 (최신 C++)." Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/smart-pointers-modern-cpp?view=msvc-170>
- 2) Microsoft. "포인터 (C++)." Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/pointers-cpp?view=msvc-170>
- 3) Microsoft. "new 및 delete 연산자." Microsoft Learn, <https://learn.microsoft.com/ko-kr/cpp/cpp/new-and-delete-operators?view=msvc-170#--the-delete-operator>
- 4) 위키독스. "02.02.02. 스마트포인터(Smart Pointer)." C++ 이야기(A Story of C++). <https://wikidocs.net/117594>
- 5) 위키독스. "02.02.02. 스마트포인터(Smart Pointer)." C++ 이야기(A Story of C++). <https://wikidocs.net/117594>
- 6) 모두의 코드. "13 - 2. 자원을 공유할 때 - shared_ptr 와 weak_ptr." 씹어먹는 C++ <https://modoocode.com/252>
- 7) 망나니개발자. "[Java] Garbage Collection(가비지 컬렉션)의 개념 및 동작 원리 (1/2)" 티스토리 <https://mangkyu.tistory.com/118>
- 8) Oracle. "HotSpot Virtual Machine Garbage Collection Tuning Guide." Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/javase/21/gctuning/introduction-garbage-collection-tuning.html>
- 9) 원정. "[C++] 스마트 포인터(Smart Pointer)란? (Raw Pointer와의 차이)." 원정의 블로그 <https://wonsjung.tistory.com/574>